WUG 2026-03-11: Warewulf & ZFS - Building a Hybrid Stateful/Stateless Cluster

Josh Burks
Senior HPC Systems Analyst
Research Technology Office
Arizona State University

Presented at WUG, March 11, 2026

ASU Research Computing
Arizona State University

Copyright © 2021 Arizona Board of Regents

Who is

Josh Burks

- Entire background in IT
- Previously worked at Intel (Manufacturing Automation)
- Been with ASU Research Computing for 3 years
- Focus on platforms – hardware, provisioning, configuration management, virtualization, storage
- Just a guy who likes to talk about Warewulf (and ZFS!)

Copyright © 2021 Arizona Board of Regents

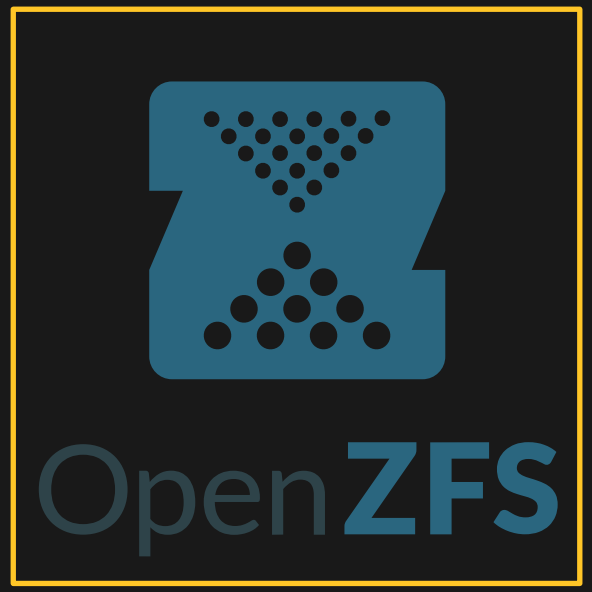
A Brief Overview Of

Warewulf @ ASU

4 Custers, ~1000 Nodes

- Sol – 287 Nodes (Flagship Cluster, Top500 – #484, Green500 – #77)
- Phoenix – 545 Nodes (Heterogeneous Cluster)
- Aloe – 22 Nodes (Secure Environment)
- New AI Cluster – 220 Nodes (Gaudi 2s running K8s)

Copyright © 2021 Arizona Board of Regents



A Brief Overview Of

ZFS

Developed by Sun Microsystems in the early 2000s, acquired by Oracle in 2009

OpenZFS is the open-source implementation of ZFS

Not part of the Linux kernel due to potential licensing conflicts between CDDL and GPL, and fears of Oracle lawsuits

ZFS combines software RAID with a volume manager, supporting snapshots, dedup, replication, self healing, compression, and automatic mounting

tldr: ZFS is awesome

Copyright © 2021 Arizona Board of Regents

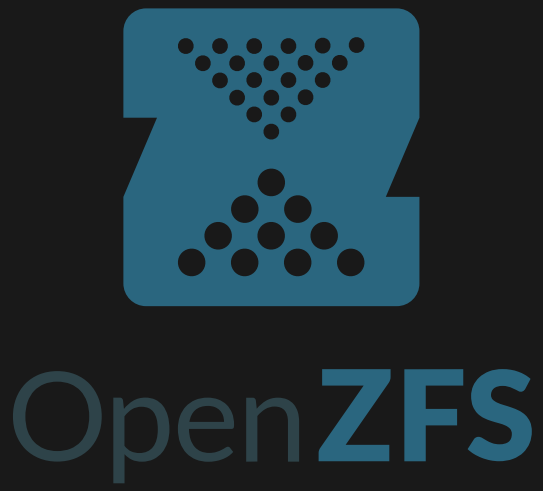
Warewulf + ZFS

Warewulf-booted nodes load the OS into memory from the network before touching disks – we can load ZFS kernel modules and configure pools during boot

Like hardware raid, ZFS configuration lives entirely on disk, nothing *needs* to stay persistent on the OS

ZFS will handle all the mounting for us, no need for /etc/fstab entries or statically defining disk configurations in nodes.conf

Vastly enhances stateful/stateless hybrid nodes



Copyright © 2021 Arizona Board of Regents

Why use a Hybrid Approach?

- Storing large rootfs in tmpfs wastes RAM
- AI workloads benefit from persistent local cache
- Kubernetes expects persistent per-node key files
- Managing per-node files with overlays is a burden
- Partitioning disks is rigid and painful
- We want memory efficiency + flexible persistence

Copyright © 2021 Arizona Board of Regents

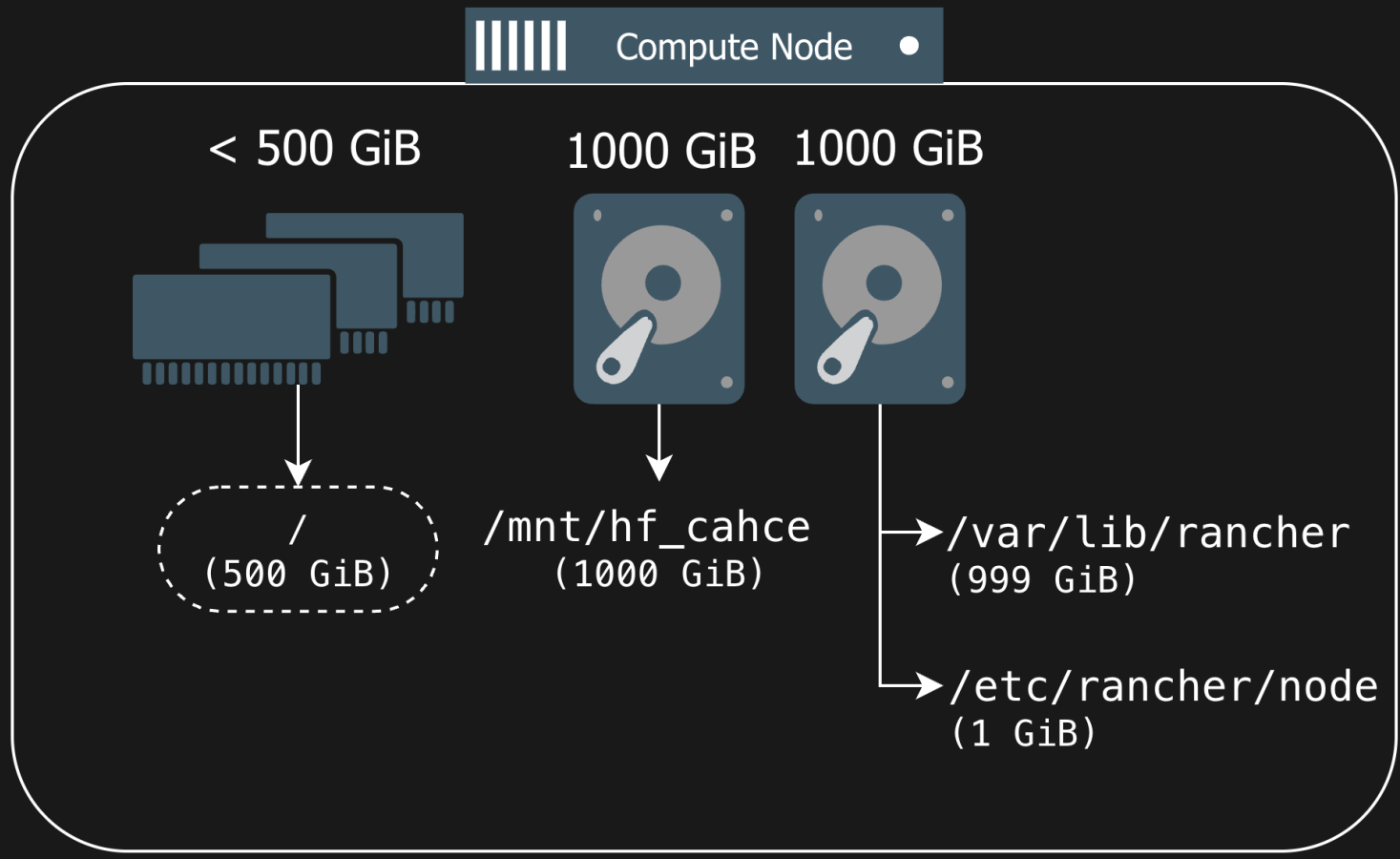
Hybrid - Stateful Data / Stateless OS

Example: A node has two 1 TiB disks, and 3 directories need to persist across reboots

The traditional way:

- rootfs in RAM (stateless)
- Disk 1 has 1 partition
- Disk 2 has 2 partitions

- Wasted RAM
- No elasticity



Copyright © 2021 Arizona Board of Regents

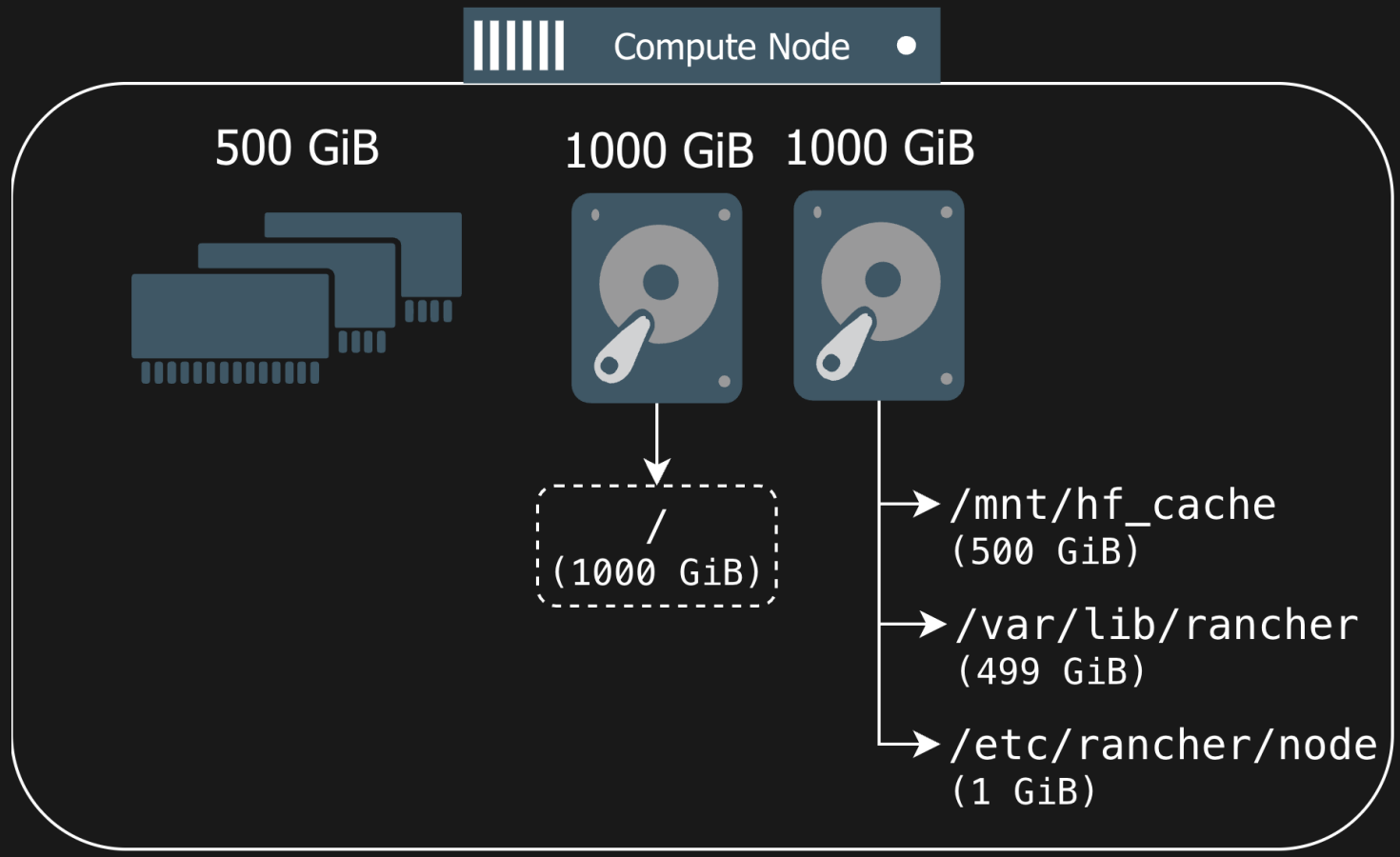
Hybrid - Stateful Data / Stateless OS

Example: A node has two 1 TiB disks, and 3 directories need to persist across reboots

The RAM-saving way:

- rootfs on disk 1 (stateless)
- disk 2 has 3 partitions

- Wasted disk space
- No elasticity



Copyright © 2021 Arizona Board of Regents

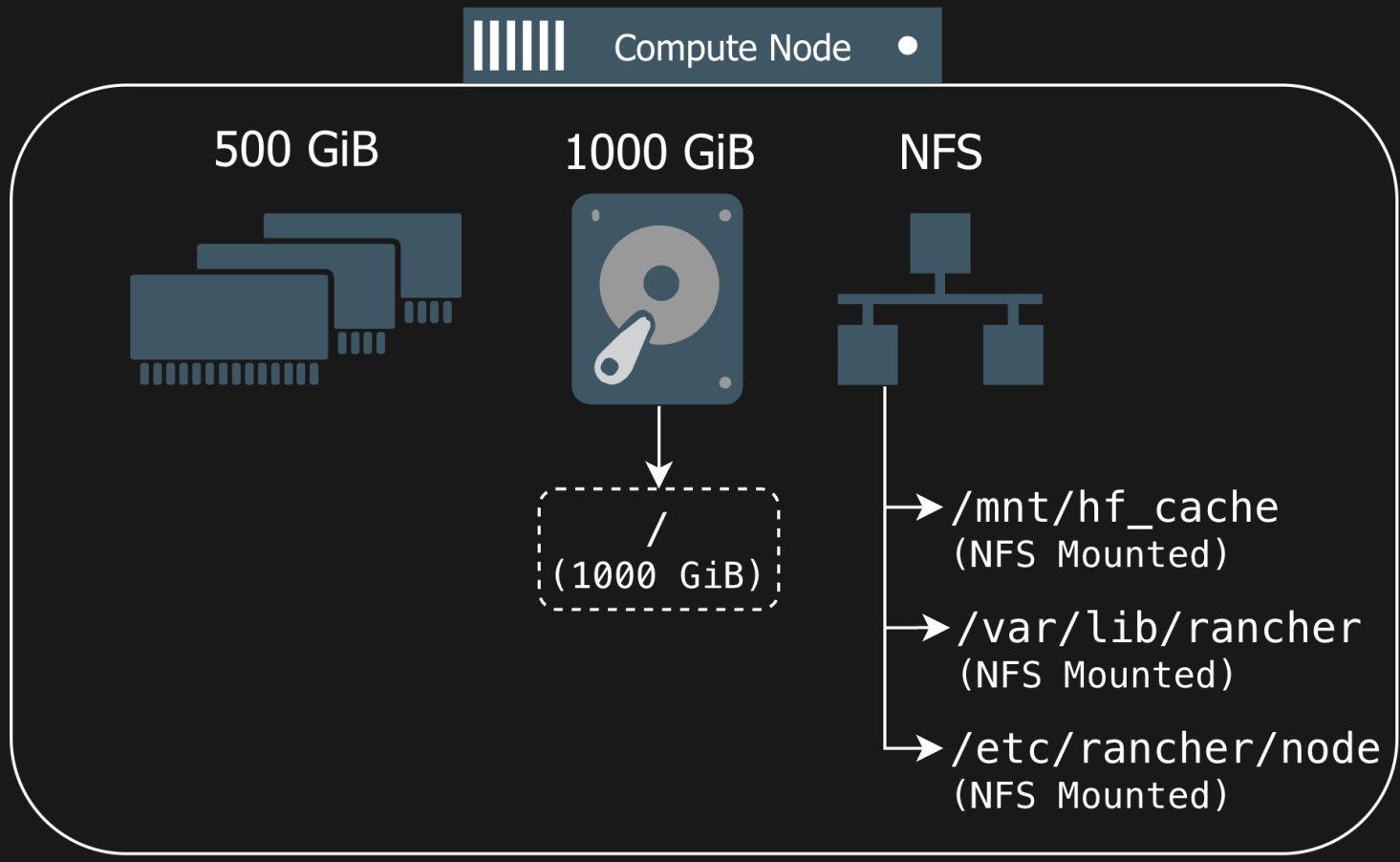
Hybrid - Stateful Data / Stateless OS

Example: A node has two 1 TiB disks, and 3 directories need to persist across reboots

The NFS way:

- rootfs on disk 1 (stateless)
- NFS mount everything else

- Higher latency
- Complicated per-node mounts



Copyright © 2021 Arizona Board of Regents

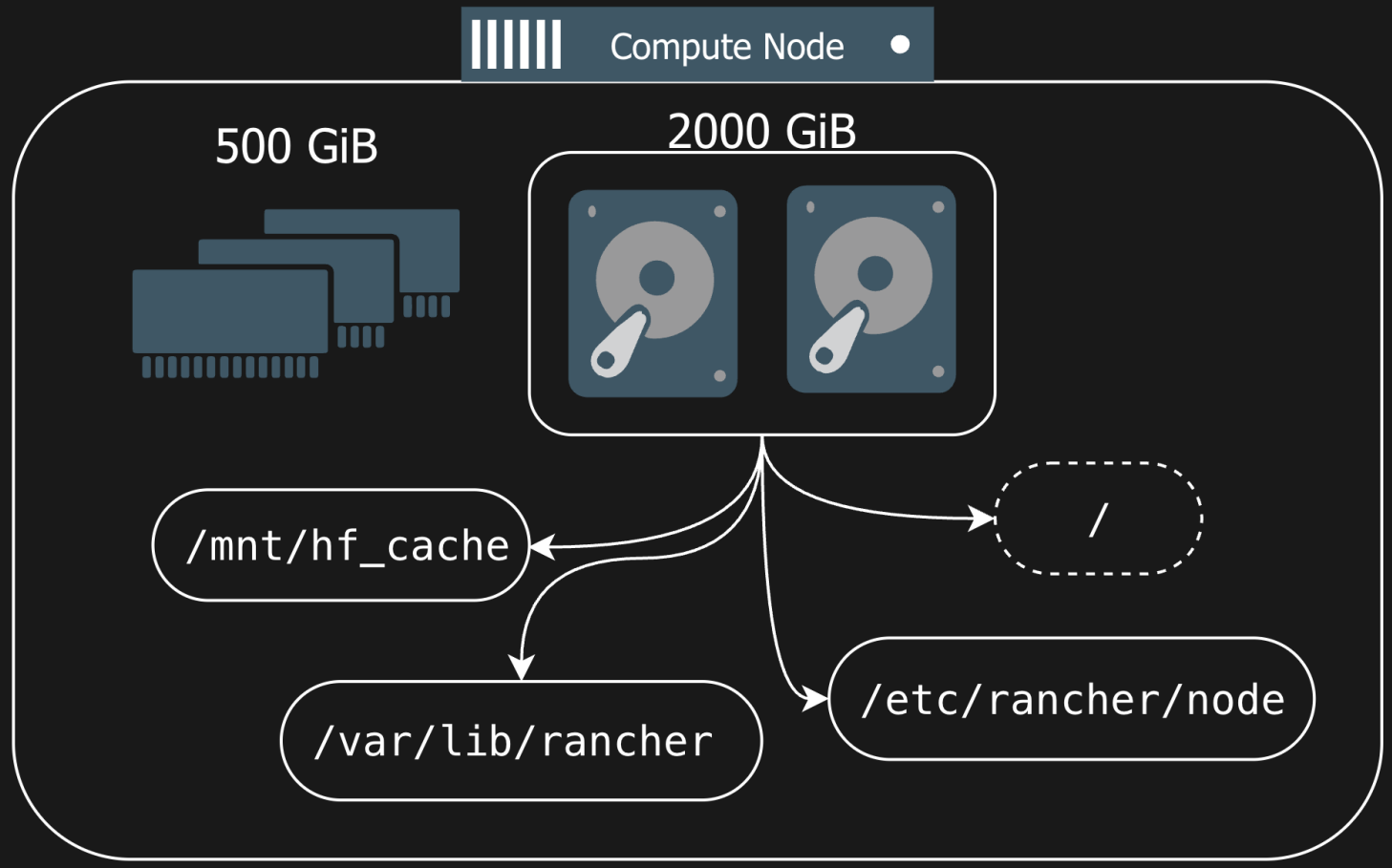
Hybrid - Stateful Data / Stateless OS

Example: A node has two 1 TiB disks, and 3 directories need to persist across reboots

The ZFS way:

- Everything on one zpool
- rootfs dataset is stateless

- No wasted disk space
- No wasted memory
- Perfect elasticity



Copyright © 2021 Arizona Board of Regents

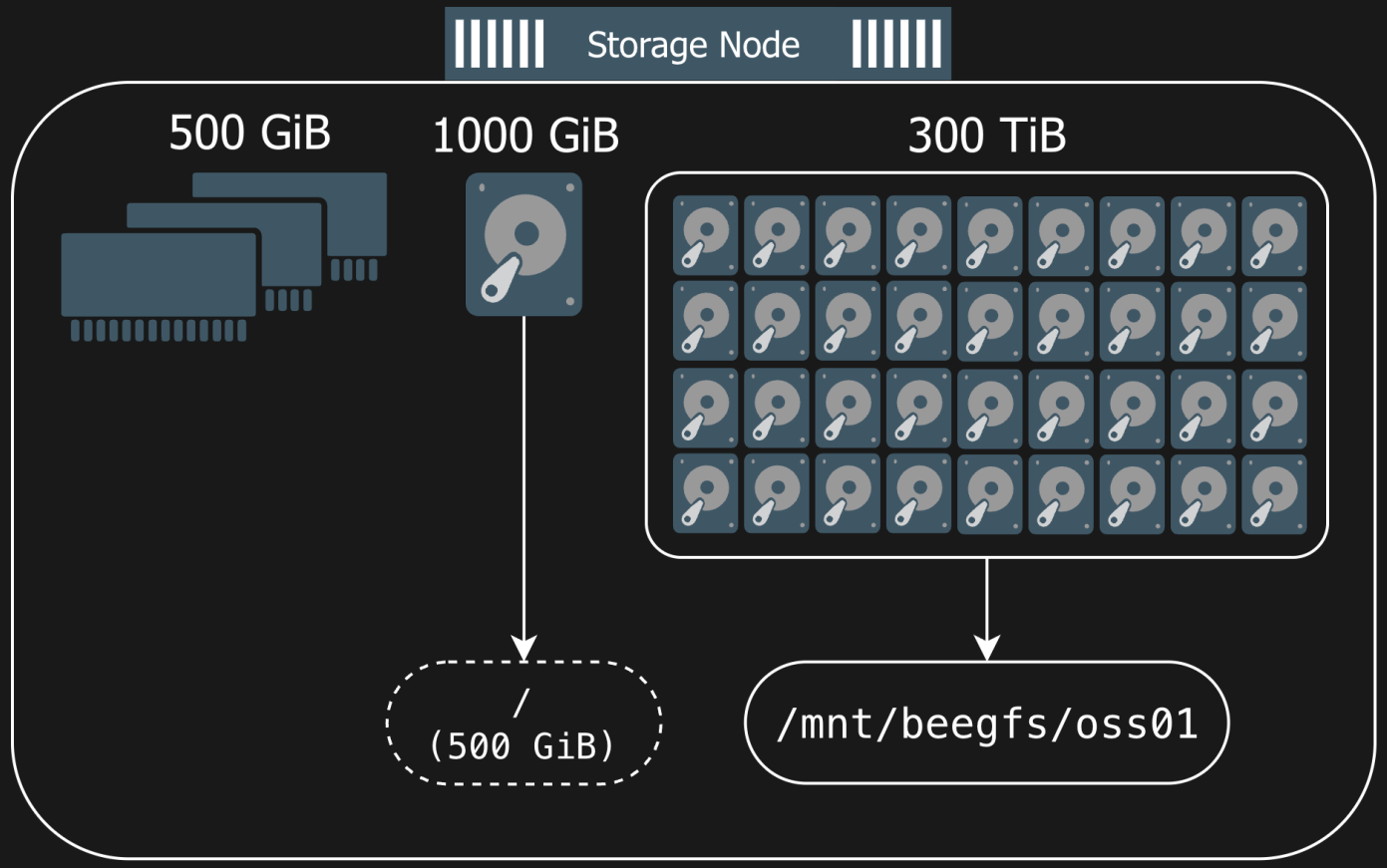
Hybrid - Stateful Data / Stateless OS

Example: A storage node needs to store persistent user data

The ZFS way:

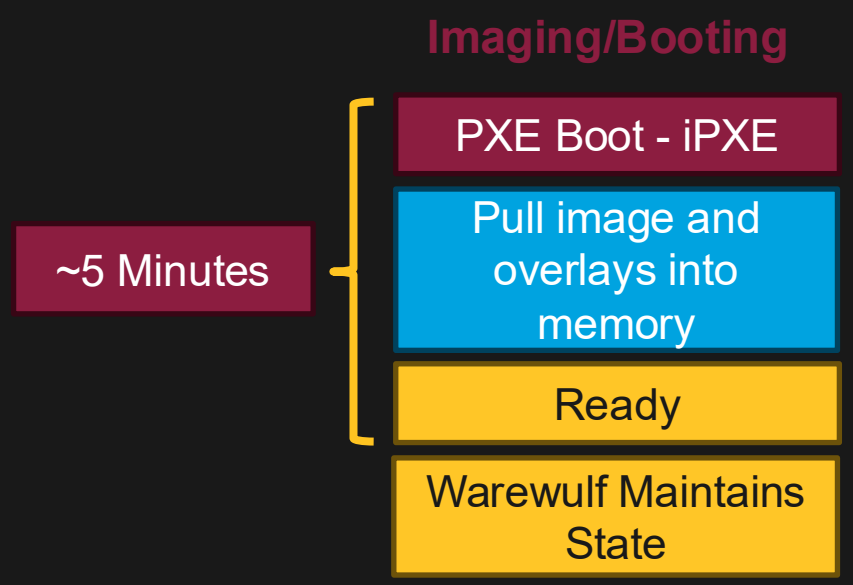
- All storage on one zpool
- Disk 1 holds stateless rootfs

- No wasted disk space
- No wasted memory



Copyright © 2021 Arizona Board of Regents

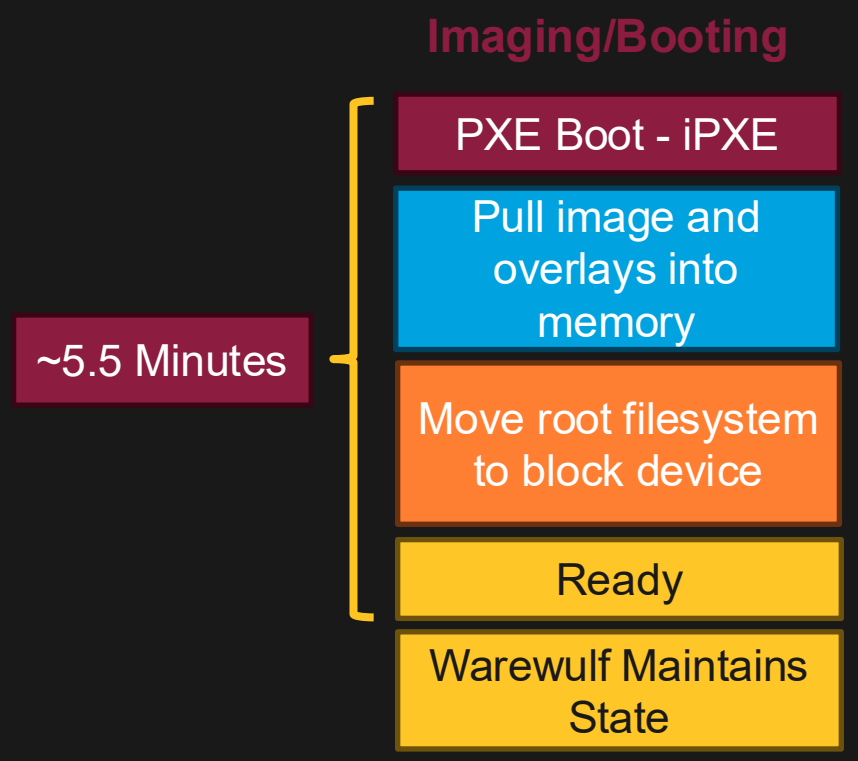
Stateless Provisioning with Warewulf



Copyright © 2021 Arizona Board of Regents

Stateless Provisioning with Warewulf

Utilizing disks to reduce memory footprint



Copyright © 2021 Arizona Board of Regents

Measuring Memory Metrics

```
WWROOT=tmpfs

df -h /
Filesystem      Size  Used Avail Use% Mounted on
wwroot           47G  7.9G   39G  17% /

free -h
              total        used        free      shared  buff/cache   available
Mem:           93Gi       1.0Gi        84Gi       7.8Gi       7.9Gi        83Gi
Swap:            0B          0B          0B
```

83 GiB Free

```
WWROOT=xfs

df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        56G  8.4G   48G  15% /

free -h
              total        used        free      shared  buff/cache   available
Mem:           93Gi       957Mi        91Gi        13Mi       453Mi        91Gi
Swap:            0B          0B          0B
```

91 GiB Free

Up to ~9.6% more memory available for jobs

Copyright © 2021 Arizona Board of Regents

Measuring Memory Metrics

1052 Nodes totaling 355 TiB RAM	X	~ 10 GiB WW Image in RAM	=	WW Images use 10.3 TiB RAM (2.9%)
1052 Nodes totaling 355 TiB RAM	X	~ 10 GiB WW Image on Disk	=	WW Images use 0 RAM

By moving the rootfs to disk, we are giving 10 TiB of memory back to researchers

Copyright © 2021 Arizona Board of Regents

Official Stateless Provisioning to Disk with Warewulf

Using 2 stage boot with dracut

Imaging/Booting

~5.5 Minutes

- PXE Boot - iPXE
- Pull kernel and initramfs into memory
- Dracut configures block devices
- Pull image and overlays to disk
- Ready
- Warewulf Maintains State

via ignition or sfdisk+mkfs

Configure the `sfdisk` and `mkfs` overlays using resources:

```
wwctl node set wwnode1 \
  --diskname /dev/vda --diskwipe \
  --partname rootfs --partcreate --partnumber 1 \
  --fsname rootfs --fsformat ext4 --fspath /
```

https://warewulf.org/docs/v4.6.x/nodes/disks.html#provision-to-disk

Copyright © 2021 Arizona Board of Regents

Official Stateless Provisioning to Disk with Warewulf

Using 2 stage boot with dracut

Imaging/Booting

~5.5 Minutes

- PXE Boot - iPXE
- Pull kernel and initramfs into memory
- Dracut configures block devices
- Pull image and overlays to disk
- Ready
- Warewulf Maintains State

arbitrary scripts in /warewulf/wwinit.d

```
if [ -x /tmp/wwinit/warewulf/run-wwinit.d ]; then
        PREFIX=/tmp/wwinit /tmp/wwinit/warewulf/run-wwinit.d
fi

info "warewulf: mounting ${wwinit_root_device} at ${NEWROOT}"
```

https://github.com/warewulf/warewulf/blob/main/dracut/modules.d/90wwinit/load-wwinit.sh

Copyright © 2021 Arizona Board of Regents

Configuring ZFS In Dracut

```
#/warewulf/wwinit.d/zfs.sh (part 1/2)

POOL="wwpool" ROOT_DS="wwroot"

# Try to import all pools but do not mount
zpool import -a -f -N

# If pool doesn't exist, create it
[ $(zpool list -H "$POOL" ) ] || create_zfs_pool

# If root dataset exists, destroy it
zfs destroy -r -f "${POOL}/${ROOT_DS}"

# Create the root dataset
zfs create -o mountpoint=legacy "${POOL}/${ROOT_DS}"
...
```

Copyright © 2021 Arizona Board of Regents

Configuring ZFS In Dracut

```
#/warewulf/wwinit.d/zfs.sh (part 2/2)
...
declare -A DATASETS=( [hf_cache]="/mnt/hf_cache"
                      [k3s]="/var/lib/rancher/k3s"
                      [rancher]="/etc/rancher/node"
                    )

# Create other datasets and set properties
for ds in "${!DATASETS[@]}"; do
  mp="${DS[$ds]}"
  if ! zfs list -H "${POOL}/${ds}"; then
    zfs create -u -o mountpoint="$mp" "${POOL}/${ds}"
    set_props "${dataset}"
  fi
done

exit 0
```

Copyright © 2021 Arizona Board of Regents

Configuring ZFS In Dracut

```
#/warewulf/wwinit.d/zfs.sh (create zfs pool function)

# Dynamically find the first two 1TiB NVMe disks and create a pool
create_zfs_pool() {
  for nvme_dev in /dev/nvme*n1; do
    size_gb=$(lsblk -bdno SIZE "$nvme_dev")
    euid="/dev/disk/by-id/nvme-$(lsblk -dno WWN "$nvme_dev")”
    # Check if it's approximately 1TiB (800-1200GB range)
    [ "$size_gb" -gt 800 ] && [ "$size_gb" -lt 1200 ] || continue
    [ -e "$euid" ] && found_disks+=("$euid")
    [ ${#found_disks[@]} -eq 2 ] && break
  done

  [ ${#found_disks[@]} -lt 2 ] && die "Could not find two 1TiB NVME disks."
  DEV0="${found_disks[0]}"; DEV1="${found_disks[1]}”

  zpool create -f -m none "${POOL}" "${DEV1}" "${DEV0}"
}
```

Copyright © 2021 Arizona Board of Regents

Booted Example – AI Cluster

```
[root@dcx-gaudi001:~]# zfs list
NAME                 USED  AVAIL  REFER  MOUNTPOINT
wwpool               577G  1.12T    96K  none
wwpool/containerd     96K  1.12T    96K  /var/lib/rancher/k3s/agent/containerd/io.containerd.snapshotter.v1.zfs
wwpool/hf_cache      566G  1.12T   566G  /mnt/hf_cache
wwpool/k3s          3.75G  1.12T  3.75G  /var/lib/rancher/k3s
wwpool/rancher        96K  1.12T    96K  /etc/rancher/node
wwpool/wwroot       7.28G  56.7G  7.28G  legacy
[root@dcx-gaudi001:~]# zfs get compressratio
NAME                PROPERTY       VALUE  SOURCE
wwpool              compressratio  1.01x  -
wwpool/containerd   compressratio  1.00x  -
wwpool/hf_cache     compressratio  1.00x  -
wwpool/k3s          compressratio  1.57x  -
wwpool/rancher      compressratio  1.01x  -
wwpool/wwroot       compressratio  1.74x  -
[root@dcx-gaudi001:~]#
```

Copyright © 2021 Arizona Board of Regents

Booted Example – Storage Node

```
[root@talon-oss01:~]# systemctl status zfs-import-scan.service
● zfs-import-scan.service - Import ZFS pools by device scanning
     Loaded: loaded (/usr/lib/systemd/system/zfs-import-scan.service; enabled; preset: disabled
     Active: active (exited) since Fri 2026-01-09 15:32:05 MST; 1 month 30 days ago
       Docs: man:zpool(8)
    Process: 2030 ExecStart=/sbin/zpool import -aN -o cachefile=none $ZPOOL_IMPORT_OPTS (code=e
   Main PID: 2030 (code=exited, status=0/SUCCESS)
        CPU: 478ms

Jan 09 15:32:01 talon-oss01 systemd[1]: Starting Import ZFS pools by device scanning...
Jan 09 15:32:05 talon-oss01 systemd[1]: Finished Import ZFS pools by device scanning.
[root@talon-oss01:~]# zfs list
NAME                    USED  AVAIL  REFER  MOUNTPOINT
storage_pool           14.4T   214T   219K  none
storage_pool/storage01 14.4T   214T  14.4T  /mnt/beegfs/storage01
[root@talon-oss01:~]#
```

```
[root@talon-oss01:~]# zpool status
  pool: storage_pool
 state: ONLINE
config:

        NAME                        STATE     READ WRITE CKSUM
        storage_pool                ONLINE       0     0     0
          raidz2-0                  ONLINE       0     0     0
            wwn-0x5000cca3c4c0b4d7  ONLINE       0     0     0
            wwn-0x5000cca251ce57ed  ONLINE       0     0     0
            wwn-0x5000cca3c4c0d206  ONLINE       0     0     0
            wwn-0x5000cca3c4c08bfd  ONLINE       0     0     0
            wwn-0x5000cca3c4c09ec7  ONLINE       0     0     0
            wwn-0x5000cca3c4c0a8af  ONLINE       0     0     0
            wwn-0x5000cca256d9963c  ONLINE       0     0     0
            wwn-0x5000cca256dac974  ONLINE       0     0     0
            wwn-0x5000cca251dc02ca  ONLINE       0     0     0
            wwn-0x5000cca251db933e  ONLINE       0     0     0
            wwn-0x5000cca256d924eb  ONLINE       0     0     0
          raidz2-1                  ONLINE       0     0     0
            wwn-0x5000cca3c4c0a674  ONLINE       0     0     0
            wwn-0x5000cca3c4c0aebe  ONLINE       0     0     0
            wwn-0x5000cca3c4c0b678  ONLINE       0     0     0
            wwn-0x5000cca3c4c09e3b  ONLINE       0     0     0
            wwn-0x5000cca3c4c12e25  ONLINE       0     0     0
            wwn-0x5000cca3c4c09a4b  ONLINE       0     0     0
            wwn-0x5000cca251dc1d10  ONLINE       0     0     0
            wwn-0x5000cca251dc1827  ONLINE       0     0     0
            wwn-0x5000cca256dba5df  ONLINE       0     0     0
            wwn-0x5000cca251dbd107  ONLINE       0     0     0
            wwn-0x5000cca256d99311  ONLINE       0     0     0
          raidz2-2                  ONLINE       0     0     0
            wwn-0x5000cca3c4c045e8  ONLINE       0     0     0
            wwn-0x5000cca3c4c0c211  ONLINE       0     0     0
            wwn-0x5000cca3c4c0d671  ONLINE       0     0     0
            wwn-0x5000cca3c4c0e12a  ONLINE       0     0     0
            wwn-0x5000cca3c4c13a16  ONLINE       0     0     0
            wwn-0x5000cca3c4c0aade  ONLINE       0     0     0
            wwn-0x5000cca26bdcead8  ONLINE       0     0     0
            wwn-0x5000cca3c4c026db  ONLINE       0     0     0
            wwn-0x5000cca251dc174c  ONLINE       0     0     0
            wwn-0x5000cca256daad5f  ONLINE       0     0     0
            wwn-0x5000cca251db6e45  ONLINE       0     0     0
        cache
          wwn-0x5001173100ccbf14    ONLINE       0     0     0
          wwn-0x5001173100b989bc    ONLINE       0     0     0
          wwn-0x5001173100ccbdbc    ONLINE       0     0     0
          wwn-0x5001173100ccc0f8    ONLINE       0     0     0
        spares
          wwn-0x5000cca251dbe7ef    AVAIL
          wwn-0x5000cca251dc0240    AVAIL
```

Building ZFS in a Containerfile

```
# Containerfile.zfs
FROM ghcr.io/warewulf/warewulf-rockylinux:9

# Cleanup old kernels
RUN dnf -y remove $(dnf repoquery --installonly --latest-limit=-1 -q)

# Install ZFS
RUN dnf -yq install https://zfsonlinux.org/epel/zfs-release-3-0.el9noarch.rpm && \
    dnf config-manager --enable zfs-latest && \
    dnf -yq install zfs zfs-dracut && \
    zfs_version=$(rpm -q zfs --qf '%{version}\n') && \
    kernel_version=$(rpm -q kernel-core --qf '%{version}-%{release}.%{arch}\n') && \
    dkms build -m zfs/$zfs_version -k $kernel_version && \
    dkms install -m zfs/$zfs_version -k $kernel_version && \
    systemctl disable zfs-import-cache && \
    systemctl enable zfs-import-scan
```

Copyright © 2021 Arizona Board of Regents

Why this matters

- All the benefits of ZFS:
  - Compression
  - De-dupe
- All the benefits of a stateless booting
  - Minimize drift
  - Negate OS breakages
- Simpler disk management
  - Elastic capacity across datasets
  - Easy scaling
  - Flexible configurations
  - Automatic mounting
- System memory returned to researchers
  - Over 10TiB across my environment

Copyright © 2021 Arizona Board of Regents

Questions?

Contact Info

Josh Burks
Senior HPC Systems Analyst
Arizona State University
josh.burks@asu.edu

GitHub
github.com/jeburks2/warewulf-extras

Research Computing
Arizona State University